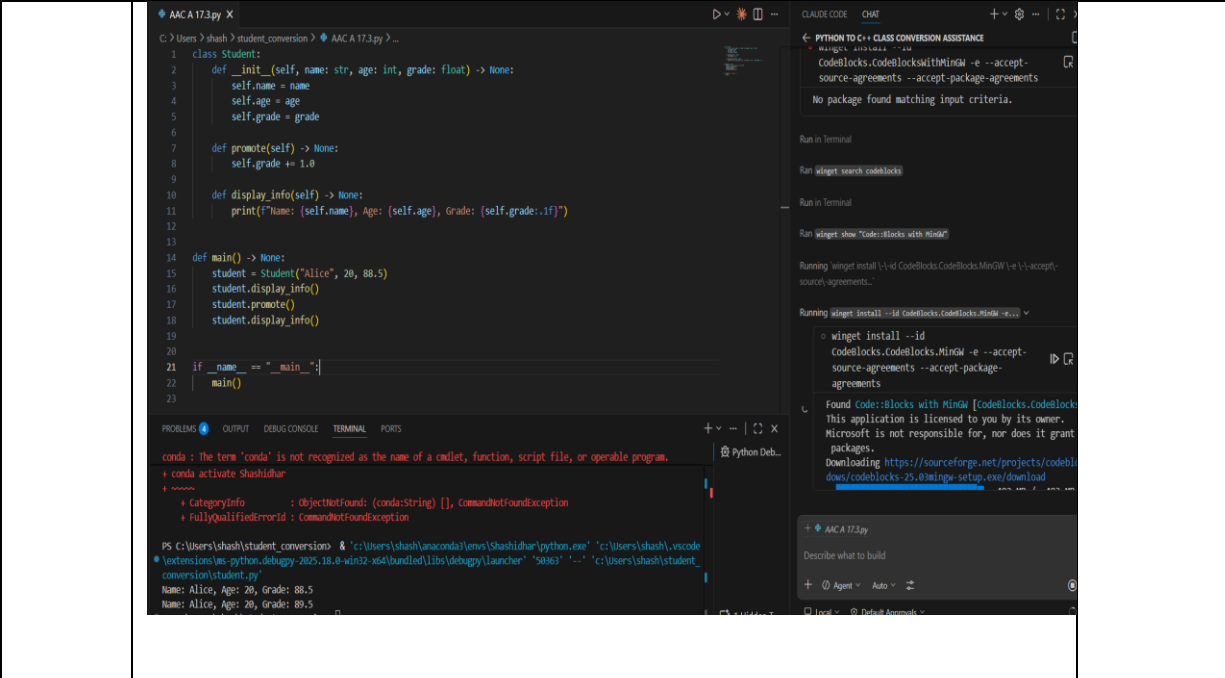


Name: J. Tejesh H.No: 2303A51812 Batch: 26

Assignment-19.3		
Q.No.	Question	Expected Time to complete
1	Lab 19 – Code Translation: Converting Between Programming Languages Lab Objectives: • Understand how AI tools can assist in translating code between different programming languages. • Learn to verify correctness and functionality after translation. • Explore syntactic and semantic differences between languages (e.g., Python, Java, C++). • Practice debugging and optimizing AI-translated code.	Week 10 - Wednesday
	Task 1 – Python to C++ Conversion Task: Translate a simple Python class into C++ using AI assistance. Instructions: • Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code • Ensure proper handling of: ◦ Constructors ◦ Data types ◦ Access specifiers • Compile and run both versions to verify output consistency Expected Output: An equivalent working C++ class translated from Python.	



Task 2 – Java to Python Function Conversion

Task:

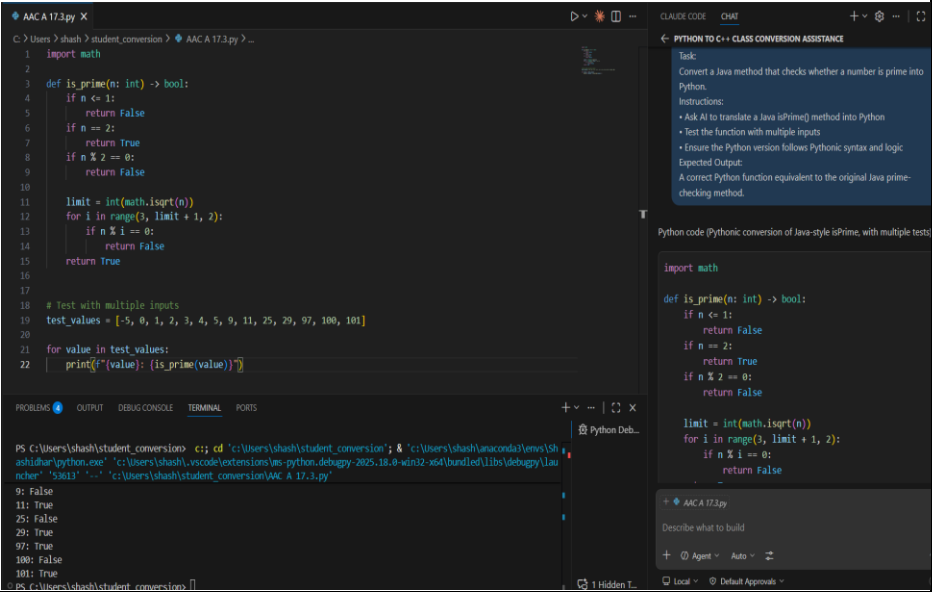
Convert a Java method that checks whether a number is prime into Python.

Instructions:

- Ask AI to translate a Java isPrime() method into Python
- Test the function with multiple inputs
- Ensure the Python version follows Pythonic syntax and logic

Expected Output:

A correct Python function equivalent to the original Java prime-checking method.



Task 3 – Pseudocode to Python Implementation

Task:

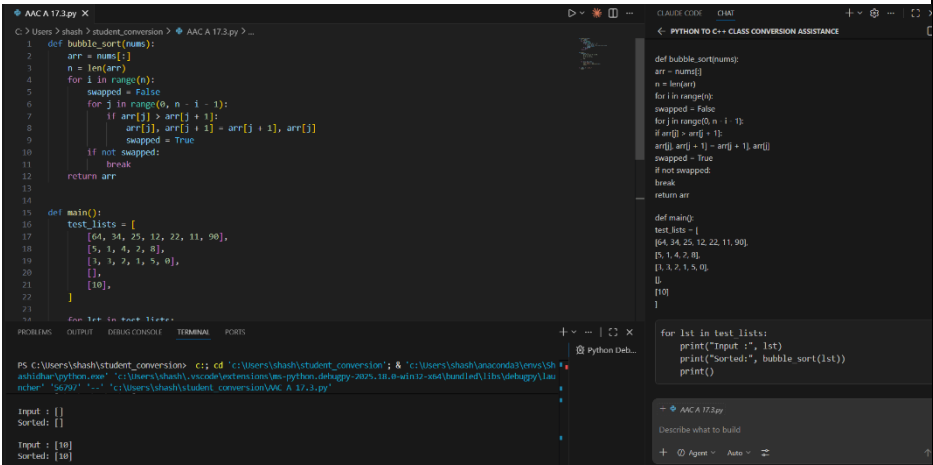
Translate a given pseudocode algorithm into Python using AI.

Instructions:

- Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
- Ask AI to convert it into executable Python code
- Validate correctness using sample input lists

Expected Output:

A working Python program that correctly implements the pseudocode logic.



```
AAC A 17.3.py X
C: > Users > shash > student_conversion > AAC A 17.3.py > ...
15 def main():
16     test_lists = [
17         [0, 1, 2, 3, 4, 5, 6, 7, 8, 9],
18         [5, 1, 4, 2, 8],
19         [3, 3, 2, 1, 5, 0],
20         [],
21         [10],
22     ]
23
24     for lst in test_lists:
25         print("Input :", lst)
26         print("Sorted:", bubble_sort(lst))
27         print()
28
29
30 if __name__ == "__main__":
31     main()
32
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\shash\student_conversion> c:; cd 'c:\Users\shash\stud
ashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.
ncher' '56797' '--' 'c:\Users\shash\student_conversion\AAC A 17.3.

Input : []
Sorted: []

Input : [10]
Sorted: [10]
```

Task 4 – SQL to Pandas Query

Task:

Translate a **SQL query** into a **Pandas DataFrame operation** in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data.

The screenshot shows a VS Code editor with a Python file named 'AAC A 17.3.py'. The script defines a Pandas DataFrame 'employees' with columns 'name', 'salary', and 'department'. It filters the DataFrame to show only employees with a salary greater than 50,000. The terminal window shows the command 'python AAC A 17.3.py' and the output of the script, which is a table with 3 rows and 2 columns: 'name' and 'salary'. The chat interface on the right shows a task description and the AI's response, which is the equivalent Pandas code.

```
1 import pandas as pd
2
3 employees = pd.DataFrame({
4     "name": ["Alice", "Bob", "Charlie", "Diana", "Evan"],
5     "salary": [48000, 62000, 51000, 45000, 75000],
6     "department": ["HR", "Engineering", "Sales", "Finance", "Engineering"],
7 })
8
9
10 result = employees.loc[employees["salary"] > 50000, ["name", "salary"]]
11
12
13 print(result)
```

```
PS C:\Users\shash\student_conversion> cd "c:\Users\shash\student_conversion" & "c:\Users\shash\anaconda3\envs\shash\python.exe" "c:\Users\shash\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher" "02723" -c "c:\Users\shash\student_conversion\AAC A 17.3.py"
PS C:\Users\shash\student_conversion> cd "c:\Users\shash\student_conversion" & "c:\Users\shash\anaconda3\envs\shash\python.exe" "c:\Users\shash\vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher" "02723" -c "c:\Users\shash\student_conversion\AAC A 17.3.py"

  name  salary
1    Bob   62000
2  Charlie  51000
4    Evan   75000
```

Task 4 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

- Equivalent Pandas query that retrieves the correct subset of data give code in python without comments

```
import pandas as pd

employees = pd.DataFrame(
    {
        "name": ["Alice", "Bob", "Charlie", "Diana", "Evan"],
        "salary": [48000, 62000, 51000, 45000, 75000],
        "department": ["HR", "Engineering", "Sales", "Finance", "Engineering"],
    }
)

result = employees.loc[employees["salary"] > 50000, ["name", "salary"]]

print(result)
```

Task 5 – Real-Time Application: Algorithm Translation Across Languages

Scenario:

A company maintains algorithms written in different programming languages.

Examples:

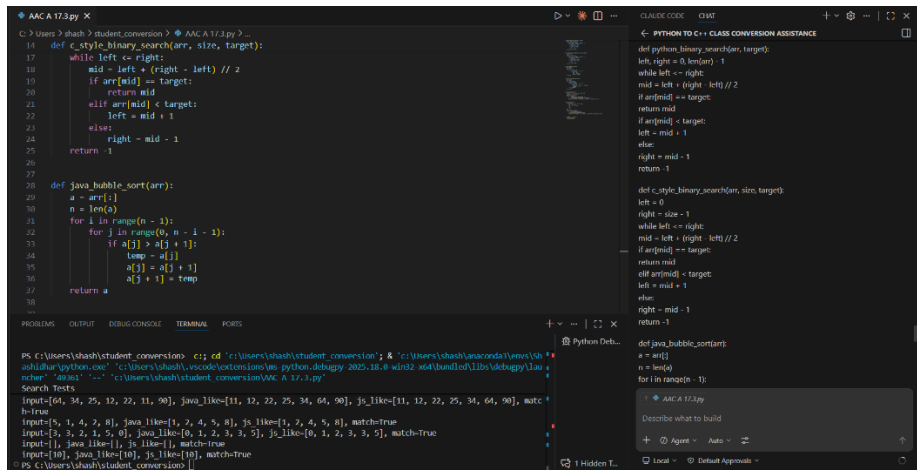
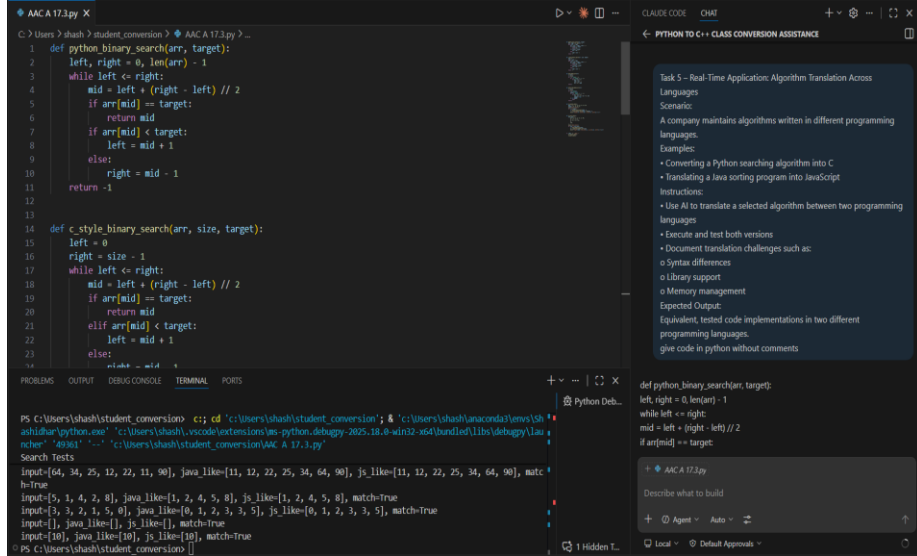
- Converting a Python searching algorithm into C
- Translating a Java sorting program into JavaScript

Instructions:

- Use AI to translate a selected algorithm between two programming languages
- Execute and test both versions
- Document translation challenges such as:
 - Syntax differences
 - Library support
 - Memory management

Expected Output:

Equivalent, tested code implementations in two different programming languages.



```
AAC A 17.3.py X
C: > Users > shash > student_conversion > AAC A 17.3.py > ...

40 def javascript_bubble_sort(arr):
41     a = arr[:]
42     n = len(a)
43     for i in range(n):
44         swapped = False
45         for j in range(0, n - i - 1):
46             if a[j] > a[j + 1]:
47                 a[j], a[j + 1] = a[j + 1], a[j]
48                 swapped = True
49         if not swapped:
50             break
51     return a
52
53
54 def run_search_tests():
55     data = [3, 7, 12, 19, 24, 31, 45, 58]
56     targets = [3, 24, 58, 9]
57     print("Search Tests")
58     for t in targets:
59         p = python_binary_search(data, t)
60         c = c_style_binary_search(data, len(data), t)
61         print(f"target={t}, python={p}, c_style={c}, match={p == c}")
62
63
64 def run_sort_tests():
65     test_cases = [
66         [64, 34, 25, 12, 22, 11, 90],
67         [5, 1, 4, 2, 8],
68         [3, 3, 2, 1, 5, 0],
69         [],
70         [10],
71     ]
72     print("\nSort Tests")
73     for case in test_cases:
74         j = java_bubble_sort(case)
75         js = javascript_bubble_sort(case)
76         print(f"input={case}, java_like={j}, js_like={js}, match={j == js}")
77
78
79 if __name__ == "__main__":
80     run_search_tests()
81     run_sort_tests()
82
```

PS C:\Users\shash\student_conversion> c++; cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '49361' '--' 'c:\Users\shash\student_conversion\AAC A 17.3.py'

Search Tests

input=[64, 34, 25, 12, 22, 11, 90], java_like=[11, 12, 22, 25, 34, 64, 90], js_like=[11, 12, 22, 25, 34, 64, 90], match=True

input=[5, 1, 4, 2, 8], java_like=[1, 2, 4, 5, 8], js_like=[1, 2, 4, 5, 8], match=True

input=[3, 3, 2, 1, 5, 0], java_like=[0, 1, 2, 3, 3, 5], js_like=[0, 1, 2, 3, 3, 5], match=True

input=[], java_like=[], js_like=[], match=True

input=[10], java_like=[10], js_like=[10], match=True

PS C:\Users\shash\student_conversion>

```
AAC A 17.3.py X
C: > Users > shash > student_conversion > AAC A 17.3.py > ...

63
64 def run_sort_tests():
65     test_cases = [
66         [64, 34, 25, 12, 22, 11, 90],
67         [5, 1, 4, 2, 8],
68         [3, 3, 2, 1, 5, 0],
69         [],
70         [10],
71     ]
72     print("\nSort Tests")
73     for case in test_cases:
74         j = java_bubble_sort(case)
75         js = javascript_bubble_sort(case)
76         print(f"input={case}, java_like={j}, js_like={js}, match={j == js}")
77
78
79 if __name__ == "__main__":
80     run_search_tests()
81     run_sort_tests()
82
```

PS C:\Users\shash\student_conversion> c++; cd 'c:\Users\shash\student_conversion'; & 'c:\Users\shash\anaconda3\envs\Shashidhar\python.exe' 'c:\Users\shash\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundle\libs\debugpy\launcher' '49361' '--' 'c:\Users\shash\student_conversion\AAC A 17.3.py'

Search Tests

input=[64, 34, 25, 12, 22, 11, 90], java_like=[11, 12, 22, 25, 34, 64, 90], js_like=[11, 12, 22, 25, 34, 64, 90], match=True

input=[5, 1, 4, 2, 8], java_like=[1, 2, 4, 5, 8], js_like=[1, 2, 4, 5, 8], match=True

input=[3, 3, 2, 1, 5, 0], java_like=[0, 1, 2, 3, 3, 5], js_like=[0, 1, 2, 3, 3, 5], match=True

input=[], java_like=[], js_like=[], match=True

input=[10], java_like=[10], js_like=[10], match=True

PS C:\Users\shash\student_conversion>

Note: Report should be submitted as a word document for all tasks in a single document with prompts, comments & code explanation, and output and if required, screenshots.